

Fiche de Route DevSecOps

Projet Crypto-Bot : De Docker à Kubernetes

1. Initialisation et Gestion de Version (Git)

Avant tout développement, le projet doit être versionné pour garder une trace des modifications et faciliter le déploiement.

Structure cible du projet (Arborescence) : Un projet bien structuré sépare le code source des configurations de déploiement (Infrastructure as Code).

```
crypto_bot/
├── fetcher/
│   ├── fetcher.py           // Script principal (logique métier)
│   ├── requirements.txt     // Dépendances Python
│   └── Dockerfile           // Recette de l'image Docker du fetcher
├── k8s/                     // Fichiers de configuration Kubernetes
│   ├── fetcher-deployment.yaml
│   ├── redis-deployment.yaml
│   └── redis-service.yaml
├── docker-compose.yml       // Configuration pour l'orchestration locale
└── .gitignore               // Règles d'exclusion Git
```

Commandes utilisées :

- `git init` : Initialise un dépôt Git local dans le dossier racine.
- `git add .` : Ajoute tous les fichiers modifiés à la « zone de préparation » (staging).
- `git commit -m "Message"` : Valide les modifications avec un message descriptif.
- `git branch -M main` : Renomme la branche principale en main.
- `git remote add origin <URL>` : Lie le dépôt local au dépôt distant (GitHub).
- `git push -u origin main` : Envoie le code vers GitHub.

Règle de Sécurité (Le fichier .gitignore) :

Il ne faut **jamais** envoyer de fichiers sensibles ou inutiles sur GitHub. On crée un fichier `.gitignore` contenant `__pycache__/, *.pyc, .env` pour empêcher Git de les traquer.

2. Le Microservice (Python & Logique)

Le cœur de l'application est le **Crypto-Fetcher**.

Explication de `fetcher.py` (sans le code) : C'est une boucle infinie qui interroge l'API de CoinGecko pour obtenir les prix du Bitcoin et de l'Ethereum, puis les écrit dans une base de données Redis. **Point clé (Résilience) :** Le code inclut une vérification du code de statut HTTP (`status_code == 429`). Si l'API nous bloque pour « Rate Limiting » (trop de requêtes par minute), le script ne crashe pas. Il intercepte l'erreur, l'affiche proprement et attend le cycle suivant.

Explication de `requirements.txt` : Liste les bibliothèques externes nécessaires (requests pour le web, redis pour la base de données).

3. Conteneurisation (Docker)

Le but est d'isoler le script Python pour qu'il tourne de manière identique sur n'importe quel ordinateur.

Structure du fichier Dockerfile : Ce fichier se lit de haut en bas et représente les différentes couches (layers) de l'image :

- FROM python:3.12-slim : Définit l'environnement OS de base (léger).
- WORKDIR /app : Crée et se place dans le dossier de travail du conteneur.
- ENV PYTHONUNBUFFERED=1 : **Astuce DevSecOps** vitale. Désactive le cache de sortie de Python pour permettre l'affichage des logs en temps réel.
- COPY requirements.txt . : Copie uniquement le fichier des dépendances en premier (optimisation du cache de build Docker).
- RUN pip install --no-cache-dir ... : Installe les bibliothèques sans garder de fichiers temporaires.
- COPY . . : Copie le reste du code source (fetcher.py).
- CMD ["python", "fetcher.py"] : La commande par défaut exécutée au démarrage du conteneur.

Commande utilisée :

- docker build -t crypto-fetcher . : Construit l'image Docker et lui donne le nom « crypto-fetcher ». Le . indique qu'il faut chercher le Dockerfile dans le dossier courant.

⚠ Piège de Répertoire :

Pour construire l'image, il faut **impérativement** se placer dans le dossier contenant le Dockerfile. `cd ~/Documents/projets/crypto_bot/fetcher`

4. Orchestration Locale (Docker Compose)

Pour faire communiquer le Fetcher et Redis sans Kubernetes, on utilise Docker Compose.

Structure du fichier docker-compose.yml : Il définit un réseau virtuel local et liste les conteneurs (appelés services) qui y tournent :

- services: : Le bloc principal contenant nos applications.
 - redis-master: :
 - image: redis:alpine : Indique qu'il faut télécharger cette image publique.
 - fetcher: :
 - build: ./fetcher : Au lieu d'une image téléchargée, on indique à Docker le chemin local vers le dossier contenant notre Dockerfile pour qu'il la construise à la volée.

Commandes utilisées :

- docker compose up --build : Construit les images (si modifié) et démarre tous les conteneurs. Affiche les logs en direct.
- Ctrl+C : Arrête proprement les conteneurs.

⚠ Piège de Répertoire :

Contrairement au build précédent, il faut lancer cette commande depuis la **racine** du projet (crypto_bot/), là où se trouve le fichier YAML.

5. Infrastructure Kubernetes : Création du Cluster (Kind)

On abandonne Docker Compose pour passer sur un vrai orchestrateur de production.

Commandes d'installation & création :

- `kind create cluster --name crypto-cluster` : Crée un cluster Kubernetes complet (Nœuds, API Server, Réseau CNI) à l'intérieur d'un grand conteneur Docker.
- `kubectl get nodes` : Vérifie que le cluster est en ligne (Statut: Ready).
- `docker start crypto-cluster-control-plane` : **Astuce vitale** pour relancer le cluster le lendemain après avoir éteint l'ordinateur, sans tout recréer.

Le transfert d'image (Le pont Docker/K8s) :

- `kind load docker-image crypto-fetcher --name crypto-cluster`



Pourquoi cette commande ? K8s tourne dans un conteneur isolé. Par défaut, il ne voit pas les images créées localement par Docker. Cette commande « téléporte » notre image `crypto-fetcher` directement dans le disque dur interne de Kubernetes. Sans cela, K8s afficherait une erreur `ErrImagePull`.

6. Déploiement sur Kubernetes

Traduction de l'architecture en objets K8s.

Structure commune des fichiers YAML Kubernetes : Dans Kubernetes, on ne donne pas des ordres directs, on déclare un « état désiré » via 4 blocs principaux :

1. `apiVersion` : La version de l'API (ex: `apps/v1` ou `v1`).
2. `kind` : Le rôle de l'objet (Deployment, Service, Pod...).
3. `metadata` : L'identité de l'objet (son name, ses labels).
4. `spec` : Les instructions précises à respecter.

1. Les Deployments (`fetcher-deployment.yaml` & `redis-deployment.yaml`) : Ce sont les « Managers ».

La partie `spec` contient :

- `replicas` : Le nombre de conteneurs (Pods) à maintenir en vie.
- `selector` : Cherche les Pods ayant un certain `matchLabels` pour s'en occuper.
- `template` : Le « moule » pour fabriquer le Pod. On y définit les containers, l'image à utiliser et les variables d'environnement (`env`).

Détail clé : Dans le `fetcher-deployment`, on ajoute `imagePullPolicy: Never` pour obliger K8s à utiliser l'image locale, évitant ainsi le téléchargement sur internet.

2. Le Service (`redis-service.yaml`) : C'est le « Standardiste » ou Routeur interne. Les Pods changent d'IP quand ils meurent, donc le Service fournit une IP fixe.

- Sa partie `spec` contient un `selector` pour repérer les Pods cibles (ex: `app: redis`), et la configuration des ports (port exposé en façade, et `targetPort` pointant vers le Pod).
- **Détail clé** : Le nom (`metadata: name: redis-master`) devient automatiquement un nom de domaine DNS interne utilisable par les autres microservices.

Commandes de gestion K8s :

- `kubectl apply -f <fichier.yaml>` : Envoie le contrat au cluster pour qu'il crée ou mette à jour l'objet.
- `kubectl get pods` : Liste les conteneurs en cours d'exécution.
- `kubectl get svc` : Liste les services (les routeurs/adresses IP internes).
- `kubectl logs -f <nom-du-pod>` : Lit les journaux du conteneur en temps réel. Utile pour déboguer l'erreur « Temporary failure in name resolution » (quand le Service DNS manquait).

7. Audit de Sécurité (Pentest Interne)

Preuve que le cluster possède des failles réseau critiques (« Flat Network » et « Zero Trust » ignoré).

L'Attaque :

- `kubectl run attacker-pod --image=redis:alpine --restart=Never -it --rm -- /bin/sh` : Lance un Pod éphémère (cheval de Troie) avec un terminal interactif. S'autodétruit à la sortie (`--rm`).
- `ping -c 3 redis-master` : Depuis le Pod attaquant, prouve que le réseau interne est ouvert (Faille 1).
- `redis-cli -h redis-master` : Se connecte à la base de données.
- `KEYS * / GET bitcoin` : Prouve qu'il n'y a aucune authentification et permet de voler les données (Faille 2).

8. Sécurisation et Remédiation (Blue Team)

Suite à l'audit, il est impératif de corriger la faille de la base de données ouverte (« Zero Trust » violé). On utilise pour cela un objet K8s sécurisé appelé Secret.

1. Création du Secret K8s : On ne stocke **jamais** un mot de passe en clair dans un fichier YAML ou sur GitHub. On crée un « coffre-fort » chiffré dans le cluster.

- `kubectl create secret generic redis-password --from-literal=password="SuperSecret123!"` : Crée le secret nommé `redis-password` contenant la clé/valeur du mot de passe.

2. Verrouillage de Redis (`redis-deployment.yaml`) : On modifie le Deployment de Redis pour lui injecter le secret en tant que variable d'environnement (`REDIS_PASS`). Ensuite, on surcharge la commande de démarrage avec `command: ["redis-server", "--requirepass", "${REDIS_PASS}"]`.

9. Mise à jour du Microservice (Gestion du Mot de Passe)

Le script Python doit maintenant s'authentifier sans compromettre la sécurité.

Modification de `fetcher.py` (Logique) : On utilise le module natif `os` pour lire la variable d'environnement (`os.environ.get("REDIS_PASS")`). On passe ensuite cette variable au paramètre `password` de la fonction de connexion `redis.Redis()`.

Mise à jour de l'image Docker (Le système de Tags) :

- `docker build -t crypto-fetcher:v2 ./fetcher` : On re-build l'image. **Bonne pratique** : on ajoute le tag `:v2` pour la différencier de l'ancienne version.
- `kind load docker-image crypto-fetcher:v2 --name crypto-cluster` : On téléporte la nouvelle version.

Mise à jour du Contrat K8s (`fetcher-deployment.yaml`) : On y ajoute l'injection du secret (bloc `env` avec `valueFrom: secretKeyRef`) pour que le Python puisse le lire, et on met à jour le nom de l'image vers `crypto-fetcher:v2`.

Commandes de déploiement :

- `kubectl apply -f fetcher-deployment.yaml`
- `kubectl rollout restart deployment crypto-fetcher-deployment` : **Commande vitale**. Elle effectue un « Rolling Update » (mise à jour progressive). K8s crée un nouveau Pod avec le nouveau code, attend qu'il soit opérationnel, puis détruit l'ancien. Résultat : l'application est mise à jour sans aucune coupure de service (Zéro Downtime).

10. Débogage Avancé (Troubleshooting)

En cas de problème (ex: erreur d'authentification persistante `NOAUTH`), un ingénieur DevSecOps doit savoir inspecter l'intérieur des conteneurs en direct.

Commandes de diagnostic chirurgical :

- `kubectl exec -it deployment/crypto-fetcher-deployment -- env | grep REDIS_PASS`

Explication détaillée :

1. `kubectl exec -it <cible>` : Ouvre un terminal interactif à l'intérieur d'un Pod géré par ce Deployment.
 2. `--` (Double tiret) : Séparateur strict. Indique à K8s que les options sont terminées et que la suite est la commande Linux à exécuter dans le conteneur.
 3. `env` : Commande listant toutes les variables système du conteneur.
 4. `| grep REDIS_PASS` : Le « Pipe » (`|`) envoie le résultat à `grep`, qui filtre et n'affiche que la ligne contenant notre mot de passe. Permet de vérifier l'injection du Secret K8s.
- `kubectl exec -it deployment/crypto-fetcher-deployment -- cat fetcher.py | grep password`
Explication : Permet de vérifier rapidement si le code Python qui tourne **réellement** dans le conteneur est bien la nouvelle version, ou si Kubernetes fait tourner une ancienne image fantôme (problème fréquent si on n'utilise pas les tags :v2).

💡 Méthodologie d'Investigation :

Si l'environnement (`env`) contient le mot de passe, l'erreur vient du code source Python ou de l'image Docker non mise à jour. Si l'environnement ne le contient pas, l'erreur vient du fichier `deployment.yaml` (erreur de syntaxe ou oubli du `kubectl apply`).

11. Déploiement et Exposition de l'API Web (Frontend)

Pour compléter notre architecture et offrir une vitrine d'affichage, nous avons introduit un troisième microservice : l'API Web (Flask). C'est le point d'entrée exposé, constituant la surface d'attaque principale (le point d'ancrage initial d'un attaquant) lors d'un audit de sécurité.

1. Structure du projet mise à jour :

Le projet adopte désormais une architecture multi-services propre.

```
crypto_bot/
├── api/                                // Nouveau dossier pour l'API Web
│   ├── api.py                        // Serveur Flask (Lecture seule dans Redis)
│   ├── requirements.txt              // Dépendances (flask, redis)
│   └── Dockerfile                   // Image Docker de l'API Web
├── fetcher/                          // (Microservice d'arrière-plan existant)
├── k8s/                              // Fichiers YAML de configuration Kubernetes
│   ├── api-deployment.yaml          // Déploiement de l'API Web
│   ├── api-service.yaml             // Service NodePort d'exposition
│   ├── fetcher-deployment.yaml
│   ├── redis-deployment.yaml
│   └── redis-service.yaml
└── docker-compose.yml
```

2. Compilation et Transfert de l'Image de l'API :

`docker build -t crypto-api:v1 ./api` : Construit l'image Docker de l'API à partir de son dossier dédié.

`kind load docker-image crypto-api:v1 --name crypto-cluster` : Téléporte l'image Docker de l'API vers le stockage interne du nœud Kubernetes géré par Kind.

12. Concepts Réseau & Exposition dans Kubernetes

Un Pod est isolé par défaut dans le cluster. Pour permettre à l'utilisateur d'accéder à l'API, nous devons configurer deux types d'objets : un Deployment pour la gestion des processus, et un Service pour l'accès réseau.

1. Le Déploiement de l'API (api-deployment.yaml) :

Gère le cycle de vie du Pod exécutant le conteneur Flask.

Configure la variable d'environnement REDIS_PASS en la lisant dynamiquement depuis le coffre-fort Secret existant (redis-password), garantissant que l'API possède les droits de lecture sans inscrire le mot de passe en clair dans le code ou sur GitHub.

2. Le Service d'Exposition Réseau (api-service.yaml) :

Pour sortir le trafic du cluster, nous utilisons le type de service NodePort. Il agit comme un mappage de ports global sur le nœud.

Port (5000) : Le port interne utilisé par les autres Pods du cluster pour contacter le service.

TargetPort (5000) : Le port sur lequel l'application Flask écoute réellement à l'intérieur de son conteneur.

NodePort (30001) : Le port externe (choisi obligatoirement entre 30000 et 32767) ouvert sur la machine physique hôte (le serveur ou ton PC) pour rediriger le trafic entrant directement vers le conteneur.

13. Commandes de Gestion et d'Investigation Réseau

En environnement de développement, certaines barrières réseau dues à la virtualisation (Docker dans Docker) exigent l'utilisation de redirections temporaires.

`kubectl apply -f k8s/api-deployment.yaml` : Déploie l'API Web sur le cluster.

`kubectl apply -f k8s/api-service.yaml` : Crée le routeur d'exposition NodePort.

`kubectl port-forward service/crypto-api-service 5000:5000` : Explication détaillée : Kind faisant tourner le cluster dans un grand conteneur Docker, le port physique de ton PC Ubuntu peut rester bloqué. Cette commande crée un tunnel direct temporaire entre le port 5000 de ton PC (localhost) et le port 5000 du Service à l'intérieur de Kubernetes. C'est l'outil indispensable pour tester son travail instantanément depuis son navigateur web.

Règle d'Or du Pentesteur (L'Analyse de Surface) :

L'API Flask utilise `host='0.0.0.0'`. Cela signifie qu'elle accepte les connexions de n'importe quel réseau. C'est indispensable pour K8s, mais cela signifie aussi que si le port du conteneur est exposé sans pare-feu (NetworkPolicy), un attaquant externe ou un conteneur voisin compromis peut attaquer l'API directement pour tenter de voler le mot de passe Redis stocké en mémoire.

14. Audit Réseau, Mouvement Latéral et Remédiation Calico (Blue Team & Red Team)

Ce volume documente l'audit de sécurité réseau de notre infrastructure, la simulation d'une intrusion applicative avec mouvement latéral, l'analyse de l'échec de notre premier pare-feu, et la reconstruction complète du cluster sous un CNI sécurisé (Calico).

14.1. 1. Le Scénario d'Intrusion & Mouvement Latéral (Red Team)

En cybersécurité, le « Pivotement » (Pivoting) consiste à compromettre un conteneur exposé en façade (l'API Flask) pour s'infiltrer ensuite dans le réseau interne privé et attaquer des cibles non exposées (la base Redis).

14.1.1. Phase A : Infiltration dans l'API Web

L'attaquant exploite une faille applicative pour obtenir un accès « shell » (ligne de commande) à l'intérieur du conteneur de l'API :

```
kubectl exec -it deployment/crypto-api-deployment -- /bin/sh
```

Explication de la commande :

`kubectl exec` : Demande à l'API Server de Kubernetes d'exécuter une commande dans un conteneur actif.

`-it` : Combine les options « `-i` » (interactif, garde l'entrée standard ouverte) et « `-t` » (alloue un terminal virtuel TTY pour formater l'affichage).

`deployment/crypto-api-deployment` : Cible le Deployment de l'API plutôt qu'un Pod individuel (ce qui évite de devoir chercher l'identifiant aléatoire du Pod).

`--` : Délimiteur strict. Indique à K8s que ses propres options s'arrêtent là et que tout ce qui suit doit être exécuté directement dans le conteneur cible.

`/bin/sh` : Le programme exécuté à l'intérieur du conteneur (ici, un interpréteur de commandes Shell).

14.1.2. Phase B : Reconnaissance et Vol de Secrets (Recon)

Une fois dans la place, l'attaquant liste les variables système en mémoire :

```
env
```

Explication : La commande `env` liste toutes les variables d'environnement. L'attaquant y découvre en clair : `REDIS_PASS=SuperSecret123!`. Cela prouve que si un conteneur est compromis, ses secrets en mémoire le sont aussi.

14.1.3. Phase C : Sabotage Réseau via Python

Puisque le conteneur n'a pas l'outil `redis-cli` installé, l'attaquant lance l'interpréteur Python natif pour manipuler Redis à distance :

```
python
```

Puis exécute le script d'exfiltration et de sabotage :

```
import redis, os
r = redis.Redis(host='redis-master', port=6379, password=os.environ.get('REDIS_PASS'),
decode_responses=True)
print(r.get('BTC'))
r.set('BTC', '999999')
exit()
```

Résultat : Le prix du Bitcoin est falsifié à 999999\$ sur le tableau de bord de l'utilisateur. Cela prouve l'absence d'isolation réseau.

14.2. 2. Le Diagnostic de l'Échec : Pourquoi le Pare-feu était inactif

Lors de notre premier essai, nous avons déployé des objets NetworkPolicy (pare-feu K8s) mais l'attaquant a quand même réussi à se connecter à Redis.

14.2.1. L'explication technique : KubeNet vs Calico

Par défaut, l'outil Kind utilise un plugin réseau (CNI) basique appelé KubeNet.

KubeNet applique un réseau « Plat » (Flat Network) où tous les Pods s'envoient des paquets de manière transparente.

KubeNet est dépourvu de moteur de filtrage. Il accepte tes fichiers YAML de sécurité mais ne les applique jamais au niveau du noyau Linux (iptables ou eBPF).

Pour avoir un pare-feu actif, il faut installer un CNI de classe entreprise comme Calico, capable de lire et d'appliquer réellement les règles réseau.

14.3. 3. Reconstruction Complète du Cluster Sécurisé (From Scratch)

Pour corriger cette faille d'infrastructure, nous devons détruire le cluster existant et le recréer avec le moteur Calico.

14.3.1. Étape A : Création du fichier de configuration Kind

On crée un fichier nommé kind-config-calico.yaml à la racine du projet crypto_bot/ :

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: crypto-cluster
networking:
  disableDefaultCNI: true
```

Explication : L'option disableDefaultCNI: true interdit à Kind d'installer son réseau par défaut (KubeNet). Le nœud démarrera mais sera inactif (NotReady) tant qu'on n'aura pas installé Calico.

14.3.2. Étape B : Séquence des commandes de reconstruction et d'installation

Destruction de l'ancien cluster vulnérable :

```
kind delete cluster --name crypto-cluster
```

Pourquoi : On ne peut pas changer de moteur réseau « à chaud ». Il faut détruire les machines virtuelles Docker et reconstruire sur des bases propres.

Recréation du cluster avec notre configuration Calico :

```
kind create cluster --config kind-config-calico.yaml
```

Pourquoi : Lit le fichier YAML pour créer un cluster sans CNI par défaut.

Déploiement du moteur de sécurité réseau Calico :

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.3/manifests/calico.yaml
```

Pourquoi : Télécharge et installe les agents Calico officiels dans le cluster. Calico va s'injecter dans le noyau Linux du nœud pour faire office de pare-feu applicatif performant.

Compilation fraîche des images Docker sur le PC Ubuntu :

```
docker build -t crypto-fetcher:v1 ./fetcher
docker build -t crypto-api:v1 ./api
```

Pourquoi : Suite à la destruction du cluster, ses images internes ont été perdues. On ré-assemble nos conteneurs sur notre hôte local.

Téléportation des images dans le nouveau cluster Kind :

```
kind load docker-image crypto-fetcher:v1 --name crypto-cluster
```

```
kind load docker-image crypto-api:v1 --name crypto-cluster
```

Pourquoi : Copie les images du PC vers l'espace de stockage interne de Kubernetes.

14.4. 4. Résolution des Pannes de Déploiement (Troubleshooting)

Lors du déploiement sur le nouveau cluster, nous avons rencontré et résolu deux erreurs critiques.

14.4.1. Panne A : L'erreur `CreateContainerConfigError` (Redis et API)

Cause : Les Pods ont refusé de démarrer car ils réclamaient le Secret `redis-password` qui avait été supprimé lors de la destruction du cluster.

Remédiation : Recréation du coffre-fort :

```
kubectl create secret generic redis-password --from-literal=password='SuperSecret123!'
```

Explication : Recrée la clé d'authentification attendue par nos fichiers YAML de déploiement.

14.4.2. Panne B : L'erreur `ErrImageNeverPull` (Fetcher)

Cause : Le YAML du Fetcher réclamait l'image `crypto-fetcher` sans préciser de tag. Kubernetes a automatiquement ajouté le tag caché `:latest`. N'ayant en mémoire locale que la version `:v1` et ayant interdiction d'aller sur Internet (Never), le Pod a planté.

Remédiation :

On force la modification de la cible d'image dans le Deployment actif :

```
kubectl set image deployment/crypto-fetcher-deployment fetcher-container=crypto-fetcher:v1
```

Explication : `set image` modifie directement le contrat Kubernetes à la volée. On associe le conteneur `fetcher-container` à la version exacte `crypto-fetcher:v1` présente dans le cluster.

On supprime le Pod défectueux pour forcer sa reconstruction immédiate :

```
kubectl delete pod crypto-fetcher-deployment-6f7c47d9bc-dvgx9
```

Explication : Le Deployment détecte la disparition du Pod, lit la nouvelle configuration d'image mise à jour (`:v1`), trouve l'image locale et démarre le nouveau Pod avec succès.

14.5. 5. Le Test de Résistance Réseau (Validation de l'Efficacité)

Pour valider que Calico bloque bien l'attaquant :

Déploiement des Network Policies de protection :

```
kubectl apply -f k8s/network-policy-deny-all.yaml
```

```
kubectl apply -f k8s/redis-network-policy.yaml
```

Lancement du Pod attaquant (sans étiquette autorisée) :

```
kubectl run attacker-pod --image=alpine --rm -it --restart=Never -- sh
```

Tentative d'assaut depuis le terminal du pirate :

```
nc -zv -w 5 redis-master 6379
```

Explication de la commande de scan :

`nc` : L'outil Netcat (le couteau suisse du réseau).

`-z` : Mode « Zéro-I/O ». Scanne le port pour voir s'il est ouvert sans envoyer de données.

`-v` : Mode verbeux. Affiche les détails de la connexion.

-w 5 : Attente maximale de 5 secondes (Timeout) avant d'abandonner.

redis-master 6379 : L'hôte cible et son port.

Résultat :

```
nc: redis-master (10.xx.xx.xx:6379): Connection timed out
```

Le pare-feu Calico a intercepté et détruit les paquets. L'accès réseau non autorisé est totalement neutralisé.

15. 6. Cartographie et Rôle des Fichiers de l'Architecture

Afin de maintenir une visibilité totale sur l'infrastructure (et de faciliter un audit ultérieur), voici la cartographie complète de notre projet ainsi que le rôle précis de chaque composant.

15.1. 6.1. Arborescence Globale du Projet

```
crypto_bot/
├── api/                                // Microservice d'Exposition (Flask)
│   ├── api.py                         // Code de l'API (Lecture seule)
│   ├── requirements.txt               // Dépendances de l'API Web
│   └── Dockerfile                     // Recette de conteneurisation de l'API
├── fetcher/                           // Microservice d'Arrière-plan (Scraper)
│   ├── fetcher.py                     // Script de collecte de cours
│   ├── requirements.txt               // Dépendances du Fetcher
│   └── Dockerfile                     // Recette de conteneurisation du Fetcher
├── k8s/                                // Dossier d'Infrastructure as Code (Kubernetes)
│   ├── api-deployment.yaml            // Déploiement de l'API Web Flask
│   ├── api-service.yaml               // Service NodePort d'exposition externe
│   ├── fetcher-deployment.yaml        // Déploiement du collecteur
│   ├── redis-deployment.yaml          // Déploiement de la base Redis durcie
│   ├── redis-service.yaml             // Routeur interne (DNS : redis-master)
│   ├── network-policy-deny-all.yaml  // Cloisonnement réseau total (Default Deny)
│   └── redis-network-policy.yaml      // White-list d'accès à Redis (Port 6379)
├── .gitignore                         // Règles d'exclusion pour Git & GitHub
├── docker-compose.yml                 // Orchestration locale pour tests rapides
└── kind-config-calico.yaml            // Fichier de provisionnement du cluster Calico
```

15.2. 6.2. Table des Matières Technique (Rôle et Action des fichiers)

Fichier	Description & Rôle d'Action Réseau
kind-config-calico.yaml	Ordonne à l'outil Kind de désactiver le réseau Kubenet basique par défaut lors du démarrage pour permettre l'injection du pare-feu Calico.
docker-compose.yml	Permet de lancer rapidement l'ensemble de l'application (Fetcher + Redis) dans un réseau Docker local hors du cluster K8s.
.gitignore	Règle de sécurité Git. Empêche l'envoi accidentel de données de cache, de secrets locaux, d'environnements virtuels ou de configurations IDE.
api/api.py	Initialise le serveur web Flask, récupère le mot de passe Redis crypté dans le système, extrait les prix de la mémoire et génère la page HTML dynamique.

api/Dockerfile	Configure l'image Docker de la vitrine web. Utilise l'astuce de cache en copiant les dépendances d'abord, et désactive le buffer de logs Python.
fetcher/fetcher.py	Boucle infinie robuste qui intercepte le Rate Limiting (erreur HTTP 429), s'authentifie sur Redis et y stocke périodiquement les prix du marché.
fetcher/Dockerfile	Construit l'image Docker du collecteur. Contient la directive vitale d'écriture des logs en temps réel pour l'investigation système.
k8s/api-deployment.yaml	Le contrat manager pour l'API. Garantit le maintien en vie de l'image locale v2 et extrait de façon transparente le mot de passe depuis le secret K8s.
k8s/api-service.yaml	Le pont d'exposition réseau. Ouvre le port 30001 sur le nœud physique de ton PC Ubuntu pour relier ton navigateur au conteneur Flask.
k8s/fetcher-deployment.yaml	Le contrat manager du Fetcher. Injecte également le secret pour que le code Python puisse s'authentifier de manière étanche.
k8s/redis-deployment.yaml	Le contrat manager de la base de données. Surcharge la commande de lancement par défaut pour exiger une authentification sécurisée par mot de passe.
k8s/redis-service.yaml	Fixe l'adresse IP interne de la base et y associe le nom DNS "redis-master" afin que les microservices puissent s'y connecter de façon fiable.
k8s/network-policy-deny-all.yaml	La première couche de sécurité réseau (Zero Trust). Bloque l'intégralité du trafic entrant pour chaque conteneur du cluster.
k8s/redis-network-policy.yaml	La seconde couche de sécurité. Crée un canal sélectif autorisant uniquement l'API Flask et le Fetcher à communiquer avec le port 6379 de Redis.